

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

File Structure Directory

stock-market-analysis/ # Main project directory (Stock Market Analysis)

|

| — my_app/ # Django application directory (if applicable)

|

|

| — _pycache_ / # Python compiled bytecode cache directory

|

|

| — migrations/ # Django database migration files

|

|

| — static/ # Static files (CSS, JavaScript, images)

|

|

|

| — css/ # CSS stylesheets

|

|

|

| — index_style.css # Styles for the index page

|

|

|

| — stock_details.css # Styles for stock details page

|

|

|

| — js/ # JavaScript files

|

|

| — search.js # JavaScript for search functionality

|

|

| — stock_details.js # JavaScript for stock details page

|

|

| — templates/ # HTML templates for the application

|

|

| — my_app/ # Templates specific to the 'my_app' application

|

|

| — forms.py # Django forms definition (if applicable)

|

|

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
| | |—— index.html    # HTML template for the index page
| | |
| | |—— stock_details.html # HTML template for stock details page
| |
| |—— __init__.py    # Marks 'my_app' as a Python package
| |
| |—— admin.py        # Django admin site configuration
| |
| |—— apps.py         # Django application configuration
| |
| |—— models.py       # Django database models
| |
| |—— tests.py        # Django unit tests
| |
| |—— urls.py         # Django URL routing for 'my_app'
| |
| |—— views.py        # Django view functions
|
|—— _pycache_/        # Python compiled bytecode cache directory
|
|—— __init__.py       # Marks 'stock-market-analysis' as a Python package
|
|—— asgi.py           # Asynchronous Server Gateway Interface file
|
|—— settings.py       # Django project settings
|
|—— urls.py           # Django project URL routing
|
|—— wsgi.py           # Web Server Gateway Interface file
|
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

- |— .gitignore # Specifies files to ignore by Git
- |
- |— db.sqlite3 # SQLite database file
- | |— manage.py # Django command-line utility
- |
- |— README.md # Project documentation
- |
- |— scripts/ # Directory for standalone scripts
- |
- |— colab_file # Python script or Jupyter Notebook for Google Colab
- |
- |— predict.py # Script for automatic predictions on all data
- |
- |— temp.py # Temporary or experimental script
- |
- |— train_colab_file.pdf # PDF report/documentation from Colab training
- |
- |— train.py # Script for training machine learning models
- |
- |— update_model.py # Script for updating/retraining models

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

1. scripts/train.py

```
// SPDX-License-Identifier: MIT

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.preprocessing import
MinMaxScaler

from sklearn.linear_model import
LinearRegression

from sklearn.metrics import
mean_squared_error, r2_score

from tensorflow.keras.models
import Sequential, load_model

from tensorflow.keras.layers
import LSTM, GRU, Dense,
Dropout

import yfinance as yf

import pickle

# Function to load stock data

def
load_stock_data(stock_symbol,
start_date, end_date):

    data =
yf.download(stock_symbol,
start=start_date,
end=end_date)[['Close', 'High',
'Low', 'Volume']]

    return data
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
# Function to prepare input data
```

```
def prepare_input_data(data,
```

```
time_steps=60):
```

```
    X, y = [], []
```

```
    for i in range(len(data) -  
time_steps):
```

```
        X.append(data[i:(i +  
time_steps)])
```

```
        y.append(data[i + time_steps,  
0]) # Predicting 'Close' price
```

```
    return np.array(X), np.array(y)
```

```
# Stock details
```

```
stock_symbol =
```

```
"RELIANCE.NS"
```

```
start_date = "2022-01-01"
```

```
end_date = "2025-02-05"
```

```
# Load and preprocess data
```

```
data =
```

```
load_stock_data(stock_symbol,  
start_date, end_date)
```

```
scaler =
```

```
MinMaxScaler(feature_range=(0,  
1))
```

```
data_scaled =
```

```
scaler.fit_transform(data)
```

```
# Save scaler
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
with open("scaler.pkl", "wb") as f:
```

```
    pickle.dump(scaler, f)
```

```
# Prepare data
```

```
time_steps = 60
```

```
X, y =
```

```
prepare_input_data(data_scaled,
```

```
time_steps)
```

```
# Split data
```

```
train_size = int(0.8 * len(X))
```

```
X_train, X_test = X[:train_size],
```

```
X[train_size:]
```

```
y_train, y_test = y[:train_size],
```

```
y[train_size:]
```

```
# Optimized LSTM model
```

```
lstm_model = Sequential([
```

```
    LSTM(128,
```

```
    return_sequences=True,
```

```
    input_shape=(time_steps, 4)),
```

```
    Dropout(0.2),
```

```
    LSTM(64,
```

```
    return_sequences=False),
```

```
    Dropout(0.2),
```

```
    Dense(32),
```

```
    Dense(1)
```

```
])
```

```
lstm_model.compile(optimizer="a
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
dam",  
loss="mean_squared_error")  
lstm_model.fit(X_train, y_train,  
epochs=50, batch_size=32,  
verbose=1)
```

```
# Save LSTM model  
lstm_model.save("lstm_model.h5  
")
```

```
# Optimized GRU model  
gru_model = Sequential([  
    GRU(128,  
return_sequences=True,  
input_shape=(time_steps, 4)),  
    Dropout(0.2),  
    GRU(64,  
return_sequences=False),  
    Dropout(0.2),  
    Dense(32),  
    Dense(1)  
)  
gru_model.compile(optimizer="a  
dam",  
loss="mean_squared_error")  
gru_model.fit(X_train, y_train,  
epochs=50, batch_size=32,  
verbose=1)
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
# Save GRU model
```

```
gru_model.save("gru_model.h5")
```

```
# Make predictions
```

```
y_pred_lstm =
```

```
lstm_model.predict(X_test)
```

```
y_pred_gru =
```

```
gru_model.predict(X_test)
```

```
# Stacking predictions for
```

```
ensemble learning
```

```
stacked_predictions =
```

```
np.hstack([y_pred_lstm,
```

```
y_pred_gru])
```

```
# Train meta-model
```

```
meta_model = LinearRegression()
```

```
meta_model.fit(stacked_predictions, y_test)
```

```
# Make final predictions using the
```

```
ensemble model
```

```
ensemble_predictions =
```

```
meta_model.predict(stacked_predictions)
```

```
# Evaluate models
```

```
mse_lstm =
```

```
mean_squared_error(y_test,
```


Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
y_pred_lstm)

mse_gru =
mean_squared_error(y_test,
y_pred_gru)

mse_ensemble =
mean_squared_error(y_test,
ensemble_predictions)


r2_lstm = r2_score(y_test,
y_pred_lstm) * 100

r2_gru = r2_score(y_test,
y_pred_gru) * 100

r2_ensemble = r2_score(y_test,
ensemble_predictions) * 100


print(f"LSTM Model Accuracy:
{r2_lstm:.2f}% (MSE:
{mse_lstm:.4f})")

print(f"GRU Model Accuracy:
{r2_gru:.2f}% (MSE:
{mse_gru:.4f})")

print(f"Ensemble Model
Accuracy: {r2_ensemble:.2f}%
(MSE: {mse_ensemble:.4f})")


# Function to plot predictions
def plot_predictions(y_test,
y_pred, model_name):

    plt.figure(figsize=(12, 6))
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
plt.plot(y_test, label="Actual  
Prices", color="blue")  
  
plt.plot(y_pred,  
label=f"{model_name}  
Predictions", color="red")  
  
plt.title(f"{model_name} Stock  
Price Prediction")  
  
plt.xlabel("Time")  
  
plt.ylabel("Stock Price")  
  
plt.legend()  
  
plt.show()
```

```
# Plot predictions  
  
plot_predictions(y_test,  
y_pred_lstm, "LSTM")  
  
plot_predictions(y_test,  
y_pred_gru, "GRU")  
  
plot_predictions(y_test,  
ensemble_predictions, "Ensemble  
Model")
```

```
# ** Predict Tomorrow's Stock  
Price **  
  
def predict_tomorrow():  
  
    last_60_days =  
data_scaled[-time_steps:] # Get  
last 60 days data  
  
    last_60_days =  
np.array([last_60_days]) #
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

Reshape for model input

```
# Get predictions

pred_lstm =
lstm_model.predict(last_60_days)
[0][0]

pred_gru =
gru_model.predict(last_60_days)[
0][0]

ensemble_input =
np.array([[pred_lstm, pred_gru]])

pred_ensemble =
meta_model.predict(ensemble_in
put)[0]
```

```
# Convert predictions back to
actual price scale

pred_lstm_actual =
scaler.inverse_transform([[pred_ls
tm, 0, 0, 0]])[0][0]

pred_gru_actual =
scaler.inverse_transform([[pred_g
ru, 0, 0, 0]])[0][0]

pred_ensemble_actual =
scaler.inverse_transform([[pred_e
nsemble, 0, 0, 0]])[0][0]
```

```
print("\nTomorrow's Predicted
Closing Prices:")
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
print(f'LSTM Model  
Prediction:  
{pred_lstm_actual:.2f} INR")  
  
print(f'GRU Model Prediction:  
{pred_gru_actual:.2f} INR")  
  
print(f'Ensemble Model  
Prediction:  
{pred_ensemble_actual:.2f}  
INR")  
  
predict_tomorrow()
```

2. scripts/predict

```
// SPDX-License-Identifier: MIT  
  
import os  
  
import numpy as np  
  
import pandas as pd  
  
import yfinance as yf  
  
import pickle  
  
from datetime import datetime,  
timedelta  
  
from tensorflow.keras.models  
import load_model  
  
from sklearn.preprocessing import  
MinMaxScaler
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
# Base directory where models  
are stored
```

```
BASE_DIR =  
r"C:\Users\sumit\Final  
year\stock-market-analysis\models"
```

```
# Get today's date
```

```
TODAY =  
datetime.today().strftime('%Y-%  
m-%d')
```

```
def  
get_latest_model_directory(stock  
_name):
```

```
    """Finds the latest date  
    directory for a given stock."""
```

```
    stock_dir =  
    os.path.join(BASE_DIR,  
    stock_name)
```

```
    if not os.path.exists(stock_dir):  
  
        print(f"No directory found  
        for {stock_name}. Skipping.")  
  
    return None
```

```
# Get all date directories and
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

sort in descending order

```
date_folders =  
sorted(os.listdir(stock_dir),  
reverse=True)
```

```
for folder in date_folders:
```

```
    # Ensure it's a valid date  
    format (YYYY-MM-DD)
```

```
    try:
```

```
        datetime.strptime(folder,  
"%Y-%m-%d")
```

```
        return  
    os.path.join(stock_dir, folder)
```

```
    except ValueError:
```

```
        continue # Skip non-date  
folders
```

```
print(f"No valid date folders  
found for {stock_name}.")
```

```
return None
```

```
def
```

```
load_models_and_scaler(stock_na  
me):
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
"""Dynamically loads the latest  
models and scaler for a given  
stock."""
```

```
latest_path =  
get_latest_model_directory(stock  
_name)
```

```
if not latest_path:
```

```
    return None, None, None,  
    None, None # Skip if no latest  
    directory found
```

```
try:
```

```
    # Load models
```

```
    lstm_model =  
load_model(os.path.join(latest_pa  
th, "lstm_model.h5"))
```

```
    gru_model =  
load_model(os.path.join(latest_pa  
th, "gru_model.h5"))
```

```
    # Load the meta-model
```

```
    with  
open(os.path.join(latest_path,  
"meta_model.pkl"), "rb") as f:
```

```
        meta_model =
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
pickle.load(f)
```

```
# Load the scaler
```

```
with
```

```
open(os.path.join(latest_path,  
"scaler.pkl"), "rb") as f:
```

```
    scaler = pickle.load(f)
```

```
    print(f"Loaded models and  
scaler from {latest_path} for  
{stock_name}")
```

```
    return lstm_model,  
gru_model, meta_model, scaler,  
latest_path
```

```
except Exception as e:
```

```
    print(f" Error loading  
models/scaler for {stock_name}:  
{e}")
```

```
    return None, None, None,  
None, None
```

```
def
```

```
load_stock_data(stock_symbol,  
time_steps=60):
```


Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
"""Load last 60 days of stock  
data from Yahoo Finance."""
```

```
try:
```

```
    data =  
    yf.download(stock_symbol,  
    period="60d",  
    interval="1d")[['Close', 'High',  
    'Low', 'Volume']]
```

```
    if data.empty:
```

```
        print(f"No data available  
for {stock_symbol}")
```

```
        return None
```

```
    return data
```

```
except Exception as e:
```

```
    print(f"Error fetching data  
for {stock_symbol}: {e}")
```

```
    return None
```

```
def prepare_input_data(data,  
scaler, time_steps=60):
```

```
    """Prepares the last 60 days of  
data for model input."""
```

```
    data_scaled =  
    scaler.transform(data[['Close',  
    'High', 'Low', 'Volume']])
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
last_60_days =  
np.array([data_scaled[-time_steps  
:]] # Prepare the last 60 days  
data  
  
return last_60_days
```

```
def calculate_range(data, pred,  
std_multiplier=0.05):
```

```
    """Calculates the range (upper  
and lower bound) for  
predictions."""
```

```
    recent_std =  
np.std(data['Close'][-60:].values)  
# Convert to NumPy array
```

```
    range_margin = recent_std *  
std_multiplier # 5% of the  
standard deviation
```

```
    lower_bound = pred -  
range_margin
```

```
    upper_bound = pred +  
range_margin
```

```
    return lower_bound,  
upper_bound
```

```
def get_prediction_date():
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
"""Returns the correct date for  
which predictions are made."""
```

```
now = datetime.now()
```

```
if now.hour < 17: # Before 5  
PM, predict for today
```

```
prediction_date =  
now.strftime('%Y-%m-%d')
```

```
else: # After 5 PM, predict for  
tomorrow
```

```
prediction_date = (now +  
timedelta(days=1)).strftime('%Y-  
%m-%d')
```

```
return prediction_date
```

```
def
```

```
predict_tomorrow(stock_symbol):
```

```
"""Predict tomorrow's stock  
price using the latest models and  
save results."""
```

```
lstm_model, gru_model,  
meta_model, scaler, latest_path =  
load_models_and_scaler(stock_sy  
mbol)
```

```
if not latest_path:
```

```
return None # Skip if no
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

models available

```
# Load the stock data for the  
last 60 days
```

```
data =  
load_stock_data(stock_symbol)
```

```
if data is None:
```

```
    return None
```

```
# Prepare input data for  
predictions
```

```
last_60_days =  
prepare_input_data(data, scaler)
```

```
# Get predictions from LSTM  
and GRU models
```

```
pred_lstm =  
lstm_model.predict(last_60_days)  
[0][0]
```

```
pred_gru =  
gru_model.predict(last_60_days)[  
0][0]
```

```
# Stack predictions and predict
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

using the ensemble model

```
ensemble_input =  
np.array([[pred_lstm, pred_gru]])
```

```
pred_ensemble =  
meta_model.predict(ensemble_in  
put)[0]
```

Convert predictions back to
actual price scale

```
def inverse_transform(value):  
  
    return  
scaler.inverse_transform([[value,  
0, 0, 0]])[0, 0] # Extract scalar  
value
```

```
pred_lstm_actual =  
inverse_transform(pred_lstm)
```

```
pred_gru_actual =  
inverse_transform(pred_gru)
```

```
pred_ensemble_actual =  
inverse_transform(pred_ensemble  
)
```

Calculate range for each
model's prediction

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
lower_bound_lstm,  
upper_bound_lstm =  
calculate_range(data,  
pred_lstm_actual)
```

```
lower_bound_gru,  
upper_bound_gru =  
calculate_range(data,  
pred_gru_actual)
```

```
lower_bound_ensemble,  
upper_bound_ensemble =  
calculate_range(data,  
pred_ensemble_actual)
```

```
# Store results in a dictionary
```

```
prediction_results = {  
  
    "Model": ["LSTM", "GRU",  
    "Ensemble"],
```

```
    "Specific Price (INR)":  
    [pred_lstm_actual,  
    pred_gru_actual,  
    pred_ensemble_actual],
```

```
    "Range": [  
  
        f"{lower_bound_lstm:.2f}  
- {upper_bound_lstm:.2f}",  
  
        f"{lower_bound_gru:.2f} -  
{upper_bound_gru:.2f}",
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
f"{lower_bound_ensemble:.2f} -  
{upper_bound_ensemble:.2f}"  
  
]  
  
}
```

```
# Convert to DataFrame
```

```
df_results =  
pd.DataFrame(prediction_results)
```

```
# Save to Excel in the same  
folder where models are stored
```

```
excel_path =  
os.path.join(latest_path,  
f"{stock_symbol}_predictions.xls  
x")
```

```
df_results.to_excel(excel_path,  
index=False)
```

```
print(f" Predictions saved for  
{stock_symbol}: {excel_path}")
```

```
# Get all stock names from  
BASE_DIR
```

```
stock_list = [d for d in  
os.listdir(BASE_DIR) if
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
os.path.isdir(os.path.join(BASE_  
DIR, d))]
```

```
# Run predictions for all stocks
```

```
for stock in stock_list:
```

```
    print(f"\nProcessing: {stock}")
```

```
    predict_tomorrow(stock)
```

3. scripts/update_model

```
import os
```

```
import numpy as np
```

```
import pandas as pd
```

```
import yfinance as yf
```

```
import pickle
```

```
from datetime import datetime,  
timedelta
```

```
from tensorflow.keras.models
```

```
import load_model
```

```
from sklearn.preprocessing import  
MinMaxScaler
```

```
from sklearn.linear_model import  
LinearRegression
```

```
import sys
```

```
import tensorflow as tf
```


Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
tf.compat.v1.experimental.output_a
```

```
ll_intermediates(True)
```

```
tf.config.run_functions_eagerly(Tru  
e)
```

```
# Base directory where models are  
stored
```

```
BASE_DIR =
```

```
r"C:\Users\sumit\Final  
year\stock-market-analysis\models"
```

```
# Get today's date
```

```
TODAY
```

```
=datetime.today().strftime('%Y-%m  
-%d')
```

```
def get_latest_folder(stock_dir):
```

```
    """Find the latest date folder  
inside the stock directory."""
```

```
    try:
```

```
        date_folders =  
sorted(os.listdir(stock_dir),  
reverse=True)
```

```
        for folder in date_folders:
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
if folder <= TODAY:
```

```
    return folder
```

```
except Exception as e:
```

```
    print(f"Error finding latest  
folder for {stock_dir}: {e}")
```

```
    return None
```

```
def load_scaler(scaler_path):
```

```
    """Load the scaler from the given  
path."""
```

```
    with open(scaler_path, 'rb') as  
scaler_file:
```

```
        return pickle.load(scaler_file)
```

```
def fetch_latest_data(stock_name):
```

```
    """Fetch the last 60 days of stock  
data, including today's data if  
available."""
```

```
    # Fetch last 90 days (to ensure  
market open days are covered)
```

```
    stock_data =  
yf.download(stock_name,  
period="90d") # Ensures at least 60
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

market days

```
if stock_data.empty:

    print(f" No data found for
{stock_name}. Skipping update.")

    return None

    # Check if the last row's date is
today

    last_row_date =
stock_data.index[-1].strftime('%Y-
%m-%d')

    if last_row_date != TODAY:

        print(f" Last row date
{last_row_date} is not today's date
{TODAY}. Skipping update.")

        return

    # Select only the required
columns

    stock_data = stock_data[['Close',
'High', 'Low', 'Volume']]

    # Get exactly the last 60
available trading days
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
if len(stock_data) < 61:
```

```
    print(f" Not enough data for  
{stock_name} (Only  
{len(stock_data)} rows). Skipping  
update.")
```

```
    return None
```

```
    return stock_data.tail(61) #
```

Ensure exactly 60 time steps

```
def retrain_models(stock_name):
```

```
    """Retrains and updates models  
    for a given stock."""
```

```
    stock_dir =  
os.path.join(BASE_DIR,  
stock_name)
```

```
    latest_folder =  
get_latest_folder(stock_dir)
```

```
    if not latest_folder:
```

```
        print(f" No previous models  
found for {stock_name}.  
Skipping.")
```

```
    return
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
latest_path =  
os.path.join(stock_dir, latest_folder)
```

```
today_path =  
os.path.join(stock_dir, TODAY)
```

```
os.makedirs(today_path,  
exist_ok=True)
```

```
print(f" Using models from  
directory: {latest_path} for  
{stock_name}") # Print Model  
Directory
```

```
# Load previous models & scaler
```

```
try:
```

```
# Fix loss function issue
```

```
lstm_model =  
load_model(os.path.join(latest_path  
, "lstm_model.h5"), compile=False)
```

```
gru_model =  
load_model(os.path.join(latest_path  
, "gru_model.h5"), compile=False)
```

```
# Explicitly compile models
```

```
lstm_model.compile(optimizer='ada
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
m', loss='mean_squared_error')
```

```
gru_model.compile(optimizer='ada
```

```
m', loss='mean_squared_error')
```

```
with
```

```
open(os.path.join(latest_path,
```

```
"meta_model.pkl"), "rb") as
```

```
meta_file:
```

```
meta_model =
```

```
pickle.load(meta_file)
```

```
scaler =
```

```
load_scaler(os.path.join(latest_path,
```

```
"scaler.pkl"))
```

```
except Exception as e:
```

```
print(f" Error loading
```

```
models/scaler for {stock_name}:
```

```
{e}")
```

```
return
```

```
# Fetch new data
```

```
stock_data =
```

```
fetch_latest_data(stock_name)
```

```
if stock_data is None or
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
stock_data.empty:
```

```
    print(f" No new data fetched  
for {stock_name}. Skipping.")  
  
    return
```

```
    stock_data_scaled =  
    scaler.transform(stock_data)
```

```
    # Extract X (features) and y  
(target)
```

```
    X = stock_data_scaled[:-1] #  
First 60 rows
```

```
    y = stock_data_scaled[-1, 0] #  
61st row's 'Close' price (Target)
```

```
    # Reshape X for LSTM  
(batch_size, time_steps, features)
```

```
    X_train = X.reshape(1, 60, 4)
```

```
    y_train = np.array([y]) # Ensure  
NumPy format
```

```
try:
```

```
    lstm_model.fit(X_train,  
y_train, epochs=50, batch_size=32,
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
verbose=1)

    gru_model.fit(X_train, y_train,
epochs=50, batch_size=32,
verbose=1)
```

```
    y_pred_lstm =
lstm_model.predict(X_train)
```

```
    y_pred_gru =
gru_model.predict(X_train)
```

```
    stacked_predictions =
np.hstack([y_pred_lstm,
y_pred_gru])
```

```
meta_model.fit(stacked_predictions
, y_train)
```

```
# Save updated models
```

```
lstm_model.save(os.path.join(today
_path, "lstm_model.h5"))
```

```
gru_model.save(os.path.join(today_
path, "gru_model.h5"))
```

```
with
```


Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
open(os.path.join(today_path,
"meta_model.pkl"), "wb") as
meta_file:

    pickle.dump(meta_model,
meta_file)

    with
open(os.path.join(today_path,
"scaler.pkl"), "wb") as scaler_file:

    pickle.dump(scaler,
scaler_file)
```

```
    print(f'Models successfully
updated for {stock_name} on
{TODAY}!')
```

```
except Exception as e:

    print(f' Error during model
training/saving for {stock_name}:
{e}')
```

```
# Get all stock names from
BASE_DIR
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
stock_list = [d for d in
os.listdir(BASE_DIR) if
os.path.isdir(os.path.join(BASE_DI
R, d))]
```

```
# Run updates for all stocks
```

```
for stock in stock_list:
```

```
    retrain_models(stock)
```

Web based application (HTML, CSS, JavaScript): -

1. static/template/my_app/index.html

```
{% load static %} <!-- Load static files -->

<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Screener</title>

    <link rel="stylesheet" href="{% static 'css/index_style.css' %}">

    <script src="{% static 'js/search.js' %}" defer></script>

</head>

<body>

    <div class="header">

        <a href="#">Home</a>

        <a href="#">Screens</a>
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

[Tools](#)

screener

Stock analysis and screening tool for investors in India.

<!-- Search box for stocks -->

[Coastal Corp](#)

[Frontier Springs](#)

[ITD Cem](#)

[Krishca Strapp.](#)

[Modison](#)

[Permanent Magnet](#)

[Pix Transmission](#)

[RACL Geartech](#)

[Sandur Manganese](#)

[Wanbury](#)

[Login](#)

[Sign Up](#)

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

</body>

</html>

2. static/template/my_app/stock_details.html

{% load static %}

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>{{ stock }} - Stock Details</title>

<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

<link rel="stylesheet" href="{% static 'css/stock_details.css' %}">

</head>

<body>

<div class="container">

<h2>{{ stock }}</h2>

<div class="price">

Current Price: {{ current_price }}
INR

({{ market_status }})

</div>

<div class="filters">

<button onclick="updateChart('1D')" class="active">1D</button>

<button onclick="updateChart('1W')">1W</button>

<button onclick="updateChart('1M')">1M</button>

<button onclick="updateChart('1Y')">1Y</button>

<button onclick="updateChart('Max')">Max</button>

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

</div>

<div class="chart-container">

<canvas id="stockChart"></canvas>

</div>

<div class="forecast">

<h3>Forecast</h3>

<label for="model">Select Model:</label>

<select id="model" class="model-select" onchange="updateForecast()">

{% for model in models %}

<option value="{{ model }}" {% if model == selected_model %}selected{%
endif %}>{{ model }}</option>

{% endfor %}

</select>

<!-- Display the Latest Range Price -->

<p>Predicted Price Range ({{ selected_model }} Model):

{% if latest_range_price %}

{{ latest_range_price }} INR

{% else %}

N/A

{% endif %}

</p>

</div>

</div>

<!-- Load JS file for handling stock data and forecast updates -->

<script src="{% static 'js/stock_details.js' %}"></script>

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

</body>

</html>

3. static/css/index_style.html

```
body {  
    font-family: Arial, sans-serif;  
    background-color: #f8f9fa;  
    text-align: center;  
    margin: 0;  
    padding: 0;  
}
```

```
.header {  
    padding: 20px;  
}
```

```
.header a {  
    margin: 0 15px;  
    text-decoration: none;  
    color: #333;  
    font-weight: bold;  
}
```

```
.container {  
    margin-top: 100px;  
}
```

```
.title {  
    font-size: 36px;  
    font-weight: bold;  
    color: #333;  
}
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
.subtitle {  
  margin-top: 10px;  
  font-size: 16px;  
  color: #666;  
}
```

```
.search-box {  
  margin-top: 30px;  
  position: relative;  
  width: 300px;  
  margin-left: auto;  
  margin-right: auto;  
}
```

```
.search-box input[type="text"] {  
  padding: 10px;  
  width: 100%;  
  font-size: 16px;  
  border: 1px solid #ccc;  
  border-radius: 4px;  
  outline: none;  
}
```

```
.search-box input[type="text"]:focus {  
  border-color: #007bff;  
}
```

```
#stockList {  
  position: absolute;  
  top: 100%;  
  left: 0;  
  width: 100%;  
  max-height: 200px;  
  overflow-y: auto;
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
border: 1px solid #ccc;
display: none; /* Hide initially */
background-color: white;
list-style-type: none;
margin: 0;
padding: 0;
z-index: 1000;
}
```

```
#stockList li {
padding: 8px;
cursor: pointer;
border-bottom: 1px solid #ddd;
}
```

```
#stockList li:hover {
background-color: #f0f0f0;
}
```

```
.suggestions {
margin-top: 20px;
}
```

```
.suggestions a {
display: inline-block;
margin: 5px 10px;
padding: 10px 15px;
background-color: #e9ecef;
border-radius: 20px;
text-decoration: none;
color: #333;
font-size: 14px;
}
```


Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
.suggestions a:hover {  
    background-color: #d6d8db;  
    color: #007bff;  
}
```

```
.top-right {  
    position: absolute;  
    top: 20px;  
    right: 20px;  
}
```

```
.top-right a {  
    text-decoration: none;  
    margin: 0 10px;  
    color: #007bff;  
}
```

```
.top-right a:hover {  
    text-decoration: underline;  
}
```

4. static/css/stock_details.css

```
/* General Styles */
```

```
body {  
    font-family: 'Poppins', sans-serif;  
    margin: 0;  
    padding: 0;  
    background: linear-gradient(to right, #1c1c1c, #2d2d2d);  
    color: #ffffff;  
}
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
/* Main Container */
```

```
.container {  
    max-width: 1000px;  
    margin: 50px auto;  
    background: #222;  
    padding: 30px;  
    border-radius: 12px;  
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.3);  
    transition: all 0.3s ease-in-out;  
}
```

```
h2 {  
    text-align: center;  
    font-size: 28px;  
    font-weight: 600;  
    margin-bottom: 15px;  
}
```

```
/* Price Display */
```

```
.price {  
    font-size: 26px;  
    font-weight: 600;  
    text-align: center;  
    margin-bottom: 20px;  
    padding: 10px;  
    background: rgba(255, 255, 255, 0.1);  
    border-radius: 8px;  
}
```

```
/* Chart Container */
```

```
.chart-container {
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
position: relative;
height: 450px;
background: #333;
padding: 20px;
border-radius: 12px;
box-shadow: inset 0 0 15px rgba(0, 0, 0, 0.5);
}

/* Filters (Time Range Buttons) */
.filters {
    text-align: center;
    margin: 25px 0;
}

.filters button {
    margin: 8px;
    padding: 10px 18px;
    border: none;
    font-size: 14px;
    font-weight: 500;
    cursor: pointer;
    background: linear-gradient(135deg, #007bff, #0044cc);
    color: white;
    border-radius: 6px;
    transition: all 0.3s ease-in-out;
}

.filters button:hover {
    background: linear-gradient(135deg, #0044cc, #003399);
    transform: scale(1.05);
}
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
.filters button.active {  
    background: linear-gradient(135deg, #ff9800, #e65100);  
    transform: scale(1.1);  
}
```

```
/* Forecast Section */
```

```
.forecast {  
    margin-top: 30px;  
    text-align: center;  
    padding: 20px;  
    background: rgba(255, 255, 255, 0.1);  
    border-radius: 8px;  
}
```

```
.forecast h3 {  
    font-size: 22px;  
    font-weight: 600;  
    margin-bottom: 10px;  
}
```

```
.model-select {  
    padding: 8px 15px;  
    font-size: 14px;  
    border-radius: 6px;  
    border: 1px solid #666;  
    background: #444;  
    color: white;  
    transition: all 0.3s ease-in-out;  
}
```

```
.model-select:hover {
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
background: #555;
```

```
}
```

```
/* Forecasted Price */
```

```
#prediction {
```

```
    font-size: 24px;
```

```
    font-weight: bold;
```

```
    color: #ff9800;
```

```
    margin-top: 10px;
```

```
    display: inline-block;
```

```
}
```

```
/* Responsive Design */
```

```
@media (max-width: 768px) {
```

```
    .container {
```

```
        padding: 20px;
```

```
    }
```

```
    .price {
```

```
        font-size: 22px;
```

```
    }
```

```
    .filters button {
```

```
        padding: 8px 12px;
```

```
        font-size: 12px;
```

```
    }
```

```
    .chart-container {
```

```
        height: 350px;
```

```
    }
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
.forecast {  
    padding: 15px;  
}  
}
```

5. static/js/search.js

```
document.addEventListener('DOMContentLoaded', () => {  
    const input = document.getElementById('stockSearch');  
    const stockList = document.getElementById('stockList');  
  
    input.addEventListener('input', () => {  
        const query = input.value;  
  
        if (query.length > 0) {  
            fetch(`/search_stocks/?query=${query}`)  
                .then(response => response.json())  
                .then(data => {  
                    // Clear previous results  
                    stockList.innerHTML = "";  
  
                    // Populate the list with new results  
                    data.forEach(stock => {  
                        const li = document.createElement('li');  
                        li.textContent = stock;
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

// Use Django URL pattern dynamically | **Language of work - Python | CSS | HTML | Javascript**

```
li.addEventListener('click', () => {
    window.location.href = `/stock/${encodeURIComponent(stock)}`;
});

stockList.appendChild(li);
});

// Show the list if there are results
stockList.style.display = data.length > 0 ? 'block' : 'none';
})
.catch(error => console.error('Error fetching stocks:', error));
} else {
    // Hide the list if query is empty
    stockList.style.display = 'none';
}
});

// Hide the stock list when clicking outside of it
document.addEventListener('click', (event) => {
    if (!input.contains(event.target) && !stockList.contains(event.target)) {
        stockList.style.display = 'none';
    }
});

// Prevent the stock list from hiding when clicking inside the input
input.addEventListener('focus', () => {
    if (stockList.childElementCount > 0) {
        stockList.style.display = 'block';
    }
});
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
});

stockSymbol = document.querySelector("h2").innerText;
let chartInstance = null;

// Fetch stock data and update the chart
function fetchStockData(range) {
  fetch(`/api/stock_data/${stockSymbol}?range=${range}`)
    .then(response => response.json())
    .then(data => {
      if (data.error || !data.timestamps || !data.prices || data.prices.length === 0) {
        console.error("Error fetching stock data or empty dataset:", data.error || "No data available");
        return;
      }

      const labels = data.timestamps; // Full timestamps for smooth graph
      const prices = data.prices;
      const xLabels = data.x_labels; // Optimized X-axis labels

      const ctx = document.getElementById('stockChart').getContext('2d');

      // Destroy previous chart if it exists
      if (chartInstance) {
        chartInstance.destroy();
      }

      chartInstance = new Chart(ctx, {
        type: 'line',
        data: {
          labels: labels,
          datasets: [{
            label: `${stockSymbol} Price`,
            data: prices,
            borderColor: '#007bff',
            backgroundColor: 'rgba(0, 123, 255, 0.1)',
            borderWidth: 2,
            pointRadius: 2,
            fill: true
          }]
        },
        options: {
          responsive: true,
          maintainAspectRatio: false,
          scales: {
            x: {
```


Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

```
display: true, Language of work - Python | CSS | HTML | Javascript
title: { display: true, text: "Time" },
ticks: {
  callback: function(value, index) {
    return xLabels.includes(labels[index]) ? labels[index] : "";
  },
  autoSkip: false,
  maxRotation: 45,
  minRotation: 0
},
y: {
  display: true,
  title: { display: true, text: "Price (INR)" }
},
elements: {
  line: { tension: 0.2 }
},
animation: {
  duration: 800
}
});
})
.catch(error => console.error("Error fetching stock data:", error));
}

// Update chart when a new range is selected
function updateChart(range) {
  document.querySelectorAll('.filters button').forEach(btn => btn.classList.remove('active'));
  document.querySelector('.filters
button[onclick="updateChart('${range}')]').classList.add('active');
  fetchStockData(range);
}

// Fetch real-time stock price
function fetchLivePrice() {
  fetch(`/api/live_price/${stockSymbol}`)
    .then(response => response.json())
    .then(data => {
      if (data.error) {
        console.error("Error fetching live price:", data.error);
        return;
      }
    })
}

document.getElementById('currentPrice').innerText = data.price || "N/A";
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

```

        document.getElementById("rangePrice").innerText = `₹${data.market_status} INR`;
    })
    .catch(error => console.error("Error fetching live price:", error));
}

// Fetch the range price for the selected model
function updateForecast() {
    let selectedModel = document.getElementById("model").value;

    // Send AJAX request to get the range price for the selected model
    fetch(`/stock/{ ${stock} }/?model=${selectedModel}`, {
        method: 'GET',
        headers: {
            'Content-Type': 'application/json'
        }
    })
    .then(response => response.json())
    .then(data => {
        if (data.range_price) {
            document.getElementById("rangePrice").innerText = `₹${data.range_price} INR`;
        } else {
            document.getElementById("rangePrice").innerText = "N/A";
        }
    })
    .catch(error => {
        console.error("Error fetching forecast:", error);
        document.getElementById("rangePrice").innerText = "Error";
    });
}

// Load initial data
fetchStockData('1D');
fetchLivePrice();
updateForecast();

// Auto-refresh live price every 30 seconds
setInterval(fetchLivePrice, 30000);

```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

6. my_app/views.py

```
from django.shortcuts import render
from django.http import JsonResponse
import yfinance as yf
import subprocess
import json
from datetime import datetime
import pytz
import requests
import numpy as np
import os
import pandas as pd
# List of 100 stocks
STOCKS = [
    'SUNPHARMA.NS', 'DRREDDY.NS', 'ITC.NS', 'ICICIBANK.NS', 'HINDUNILVR.NS',
    'ASIANPAINT.NS', 'BRITANNIA.NS', 'TCS.NS', 'PIDILITIND.NS', 'HDFCBANK.NS',
    'INFY.NS', 'RELIANCE.NS', 'BHARTIARTL.NS', 'SHREECEM.NS', 'BAJAJFINSV.NS',
    # (Add remaining stocks...)
]

def index(request):
    return render(request, 'my_app/index.html')

def search_stocks(request):
    """Search stocks dynamically"""
    query = request.GET.get('query', '').lower()
    matching_stocks = [stock for stock in STOCKS if query in stock.lower()]
    return JsonResponse(matching_stocks, safe=False)

def is_market_open():
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
"""Check if the stock market is currently open (Indian Market Hours)"""  
  
now = datetime.now(pytz.timezone("Asia/Kolkata"))  
  
return now.weekday() < 5 and now.hour >= 9 and now.hour < 15 # 9:00 AM - 3:30 PM  
IST
```

```
def get_live_price(request, stock_name):
```

```
    """Fetch live stock price or latest closing price"""
```

```
    stock = yf.Ticker(stock_name)
```

```
    data = stock.history(period='1d')
```

```
    market_status = "Closed"
```

```
    price = None
```

```
    if not data.empty:
```

```
        now = datetime.now(pytz.timezone("Asia/Kolkata"))
```

```
        market_open_time = now.replace(hour=9, minute=15, second=0, microsecond=0)
```

```
        market_close_time = now.replace(hour=15, minute=30, second=0, microsecond=0)
```

```
        if market_open_time <= now <= market_close_time:
```

```
            try:
```

```
                price = stock.history(period='1m', interval='5m').iloc[-1]['Close']
```

```
                market_status = "Open"
```

```
            except:
```

```
                price = data['Close'].iloc[-1] # Fallback to last close
```

```
        else:
```

```
            price = data['Close'].iloc[-1]
```

```
    return JsonResponse({'price': round(price, 2) if price else None, 'market_status':  
market_status})
```

```
def get_stock_data(request, stock_name):
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
"""Fetch stock price history for different time ranges with optimized X-axis intervals"""
```

```
range_option = request.GET.get('range', '1D')
```

```
stock = yf.Ticker(stock_name)
```

```
# Fetch stock history based on range
```

```
if range_option == "1D":
```

```
    data = stock.history(period="1d", interval="1m") # Intraday prices
```

```
    date_format = '%H:%M' # Only show time for intraday
```

```
elif range_option == "1W":
```

```
    data = stock.history(period="5d", interval="15m") # Hourly data over a week
```

```
    date_format = '%Y-%m-%d %H:%M'
```

```
elif range_option == "1M":
```

```
    data = stock.history(period="1mo", interval="1d") # Daily closing prices
```

```
    date_format = '%Y-%m-%d'
```

```
elif range_option == "1Y":
```

```
    data = stock.history(period="1y", interval="1d") # Daily closing prices
```

```
    date_format = '%Y-%m-%d'
```

```
else:
```

```
    data = stock.history(period="max", interval="1d") # Max available data
```

```
    date_format = '%Y-%m-%d'
```

```
if data.empty:
```

```
    return JsonResponse({'error': 'No data found'}, status=404)
```

```
timestamps = data.index.strftime(date_format).tolist()
```

```
prices = data['Close'].tolist()
```

```
# Select X-axis labels dynamically
```

```
def select_x_labels(timestamps, prices, num_labels):
```

```
    """
```

```
    Dynamically selects X-axis labels while keeping data aligned.
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

Ensures all timestamps are covered without overcrowding.

```
"""

num_data_points = len(timestamps)

if num_data_points <= num_labels:
    return timestamps, prices # Return all points if few data points exist

step = max(1, num_data_points // num_labels) # Avoid step = 0

return [timestamps[i] for i in range(0, num_data_points, step)], \
        [prices[i] for i in range(0, num_data_points, step)]

# Define number of X-axis labels dynamically
num_labels_map = {
    "1D": 6, # Show 6 spaced-out time points (e.g., every 1-2 hours)
    "1W": 6, # Show 6 spaced-out days
    "1M": 6, # Show 6 spaced-out days
    "1Y": 6, # Show 6 months
    "max": 6 # Show 6 years
}

num_labels = num_labels_map.get(range_option, 6)
x_labels, x_prices = select_x_labels(timestamps, prices, num_labels)

return JsonResponse({
    'timestamps': timestamps, # Full timestamps for chart data
    'prices': prices, # Full price data
    'x_labels': x_labels, # Optimized X-axis labels
    'x_prices': x_prices # Corresponding prices for labels
})
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
def get_forecast(request, stock_name, model):  
    """Fetch predictions from predict.py & get latest range price from Excel"""  
    try:  
        # Run the prediction script with the selected model  
        result = subprocess.run(  
            ['python', 'predict.py', stock_name, model],  
            capture_output=True, text=True  
        )  
        prediction = json.loads(result.stdout)  
  
        # Fetch latest Range price from Excel  
        latest_range_price = get_latest_range_price(stock_name, model)  
  
        # Return both predicted price & range price  
        return JsonResponse({  
            'prediction': round(prediction.get('predicted_price', 0), 2),  
            'range_price': latest_range_price if latest_range_price is not None else "N/A"  
        })  
  
    except Exception as e:  
        return JsonResponse({'error': str(e)}, status=500)  
  
# Path to stock model directory (update as needed)  
DATA_DIR = r"C:\Users\sumit\Final year\stock-market-analysis\models"  
  
def get_latest_range_price(stock_name, model_name):  
    """Find the latest date folder, read the Excel file, and return the latest 'Range' price for the  
    selected model."""  
    try:  
        # Construct the path for the stock's base directory
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
stock_dir = os.path.join(DATA_DIR, stock_name)

print(f'Checking directory: {stock_dir}')


# Ensure the stock directory exists
if not os.path.exists(stock_dir):

    print("Error: Stock directory does not exist.")

    return None # No data available


# Get the latest folder (sorted by date)
folders = [f for f in os.listdir(stock_dir) if os.path.isdir(os.path.join(stock_dir, f))]

print(f'Available date folders: {folders}')


if not folders:

    print("Error: No date folders found in stock directory.")

    return None


# Select the latest folder based on the date
latest_folder = max(folders, key=lambda d: d) # Latest date folder
folder_path = os.path.join(stock_dir, latest_folder)

print(f'Latest folder selected: {latest_folder}')


# Find the Excel file for the predictions (it will be named
<stock_name>_predictions.xlsx)
excel_files = [f for f in os.listdir(folder_path) if f.endswith('.xlsx')]

print(f'Excel files found: {excel_files}')


if not excel_files:

    print("Error: No Excel files found in the latest folder.")

    return None


# Check if the correct file exists (e.g., <stock_name>_predictions.xlsx)
```


Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
latest_file = [f for f in excel_files if f.startswith(stock_name + '_predictions')]

if not latest_file:

    print("Error: No prediction file found.")

    return None

latest_file = latest_file[0] # Use the first match

file_path = os.path.join(folder_path, latest_file)

print(f"Latest Excel file selected: {latest_file}")

# Read the Excel file

df = pd.read_excel(file_path)

print("Excel file read successfully.")

# Ensure 'Range' and 'Model' columns exist

if 'Range' not in df.columns or 'Model' not in df.columns:

    print(f"Error: 'Range' or 'Model' column not found in {file_path}")

    print(f"Available columns: {df.columns.tolist()}")

    return None

# Filter the dataframe for the selected model

model_data = df[df['Model'] == model_name]

if model_data.empty:

    print(f"Error: No data found for model {model_name}")

    return None

# Get the Range value for the selected model

latest_range = model_data.iloc[-1]['Range'] # Fetch the last row for the selected model

print(f"Latest 'Range' value for model {model_name}: {latest_range}")

# Return the range price as a string, without any modification

return latest_range
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
except Exception as e:
```

```
    print(f"Error reading Excel file: {e}")
```

```
    return None
```

```
from django.http import JsonResponse
```

```
def stock_details(request, stock_name):
```

```
    """Render stock details page with live price & model-based Range price."""
```

```
    live_price_url = f"http://127.0.0.1:8000/api/live_price/{stock_name}/"
```

```
    try:
```

```
        response = requests.get(live_price_url)
```

```
        stock_price = response.json()
```

```
    except Exception:
```

```
        stock_price = {'price': None, 'market_status': "Unknown"}
```

```
    # Get selected model from request (default: Ensemble)
```

```
    selected_model = request.GET.get('model', 'Ensemble')
```

```
    # Fetch latest Range price from Excel based on the selected model
```

```
    latest_range_price = get_latest_range_price(stock_name, selected_model)
```

```
    # If the request is an AJAX request, send back the range price
```

```
    if request.headers.get('X-Requested-With') == 'XMLHttpRequest':
```

```
        if latest_range_price:
```

```
            return JsonResponse({'range_price': latest_range_price})
```

```
        else:
```

```
            return JsonResponse({'error': 'Range price not available'}, status=400)
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
return render(request, 'my_app/stock_details.html', {  
    'stock': stock_name,  
    'current_price': stock_price.get('price'),  
    'market_status': stock_price.get('market_status'),  
    'models': ['LSTM', 'GRU', 'Ensemble'],  
    'selected_model': selected_model,  
    'latest_range_price': latest_range_price  
})
```

7. my_app/urls.py

```
import type { Metadata } from "next";  
import { Inter } from "next/font/google";
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
from django.urls import path

from . import views

urlpatterns = [

    path("", views.index, name='index'),

    path('search_stocks/', views.search_stocks, name='search_stocks'),

    path('stock/<str:stock_name>', views.stock_details, name='stock_details'), # Keep only
one stock details path


    # API Endpoints

    path('api/live_price/<str:stock_name>', views.get_live_price, name='live_price'),

    path('api/stock_data/<str:stock_name>', views.get_stock_data, name='stock_data'),


    # Updated Forecast API to support model selection

    path('api/forecast/<str:stock_name>/<str:model>', views.get_forecast, name='forecast'),

]


};
```

App Setting/Configuration

1. stock_market_analysis/setting.py

```
"""

Django settings for stock_market_analysis project.

Generated by 'django-admin startproject' using Django 4.x.

"""

import os
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
from pathlib import Path

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'your-secret-key'

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = []

# Application definition
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    # Add your apps here
    'my_app',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'stock_market_analysis.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'stock_market_analysis.wsgi.application'

# Database
# https://docs.djangoproject.com/en/4.x/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

```
}

# Password validation
# https://docs.djangoproject.com/en/4.x/ref/settings/#auth-password-validators
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]

# Internationalization
# https://docs.djangoproject.com/en/4.x/topics/i18n/
LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'

USE_I18N = True

USE_L10N = True

USE_TZ = True
```

Sentiment-Infused Stock Price Prediction: A Hybrid Model for Capturing Market Sentiments and Financial Trends

Language of work - Python | CSS | HTML | Javascript

Static files (CSS, JavaScript, Images)

<https://docs.djangoproject.com/en/4.x/howto/static-files/>

STATIC_URL = '/static/'

Default primary key field type

<https://docs.djangoproject.com/en/4.x/ref/settings/#default-auto-field>

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'